Menagerie of CS Notes

A disorganized heap of typed notes from various CS classes ranging from CS 110: Principles of Computer Systems to CS 103: Mathematical Foundation of Computing. Mostly topics I thought were interesting or important.

Contents

Intro to Filesystem & Permissions (CS110) **2**

Pointers C++ Review (CS110) **3**

Sets (CS 103) **5**
Set Theory 5

Proof Writing (CS103) **6**

Statements (CS103) **7**
Implication 7
Negation 7
Contrapositive 8
Proof by Contradiction 8

Joint Probability Density Functions (CS109) **8**

Beta Distribution (CS109) **9**

Regression (CS229) **10**

Kernel Method (CS229) **11**

Intro to Filesystem & Permissions (CS110)

The UNIX filesystem is a tree-based model to store files and directories of files.. We can retrieve a detailed listing with the **ls -al** command of the files/directories present. The "rwx——" designates the permissions of the file/directory, with r for read, w for write, and x for execute.

There are three parts to the permission line. For instance, in "rwx r-x r-x", the parts are owner, group, and user permissions, respectively. One takeaway from this is that the user has read (r) and execute (x) permissions, but not write (w).

The permissions are represented as binary strings, with "rwx" designated as 111 and "r-x" designated as 101. Hence, "rwx r-x r-x" would be represented as 111 101 101, or 755 in decimal.
A file can be created using the **open** system call, which also enables us to set the permissions. The **open** function comes with the following signatures.

```
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

There are several flags (**man 2 open** provides all) that can be combined together in *bitwise* fashion.

- **O_RDONLY**: Read only
- **O_WRONLY**: Write only
- **O_RDWR**: Read and write

If creating the file in either write only or read only mode, we must employ either **0_CREAT** to create the file if non-existent or **O_EXCL** to mandate the file be created from scratch and fail if it already exists.

The third argument mode, *in part* decides the file's permissions. There is a default permissions mask **umask**, a three-digit octal, imposing limits on the permissions. For instance, 022 means that all permission bits can be set except for the group's and other's individual write bits, as 2 is 010 in binary. The 0 digit means that no limits are imposed on owner permissions.

```
#include <stdio.h>
#include <sys/types.h>
#incude <sys/stat.h>
int main(int argc, char *argv[]){
    mode_t old_mask = umask(0);
    umask(old_mask); //restore to the original
    printf("umask is set to %03o\n",old_mask);
    return 0;
}
```

One can override **umask** if they need to set the permissions a particular a way.

```
/* File: open.c */
#include <fcntl.h> //for open
#include <unistd.h> //for read, write, close
#include <stdio.h>
#include <sys/types.h>
#incude <sys/stat.h>
#include <errno.h>

const char *kFilename = "empty";
const int kFileExistsErr = EEXIST; //constant for magic number of 17
```

```
int main(int argc, char *argv[]){
    mode_t old = umask(0); //override permissions
    int fd = open(kFilename, O_WRONLY | O_CREAT | O_EXCL, 0777);
    umask(old); //reset permissions
    if (fd == -1){ //executed if there is a failure (file already exists is a likely one)
        if (errno == kFileExistsErr){
            printf("\"%s\" already exists!!!", kFilename);
        }
        else{
            printf("Something failed!");
        }
        return 1;
    }
    close(fd);
    return 0;
}
```

Pointers C++ Review (CS110)

A binary data file has the format of several sequential bytes with a pointer to the initial byte. If the first four bytes of the sequence contain an unsigned integer **numSecrets**, we can access the value through deference.

```
unsigned int numSecrets = *(unsigned int *)secretPtr;
```

The structure of the data file can seem like (where secretPtr points to the start of numSecrets):

numSecrets 4 bytes	secretLetter 1 byte	Padding null-term 2 1 byte	secretNumbers 2 bytes	secretNumbers 2 bytes

Here, we have an array of unsigned shorts containing secret numbers. We can use the following pointer arithmetic to acquire a pointer to the first element of the array.

```
unsigned short *secretNumPtr = (unsigned short *)((char *)secretPtr + sizeof(unsigned int
    ) + 2 * sizeof(char));
unsigned short secretNum = *secretNumPtr; //dereference
```

Using (char *) allows us to scale all pointer arithmetic to sizeof(char), which is just one byte. To get the last secret number in the array of unsigned shorts, which has **numSecrets** elements, we can use the following code.

```
//pointer arithmetic is in units of sizeof(unsigned short) = 2
//the * dereferences and gets the value as an unsigned short
unsigned short lastNum = *(secretNumPtr + numSecrets - 1);

//array notation approach
unsigned short lastNum = secretNumPtr[numSecrets - 1];
```

Suppose that after the array of *secret numbers*, we have an array of null-terminated characters called the **secretWord**. A possible appproach is:

```
string secretWord c= (char *)(secretNumPtr + numSecrets);
```

C++ has a much larger set of libraries, makes it possible to build generics and classes, and easier to manufacture larger data structures.

One of the most visual and distinctive features of C++ is the design of classes. The following class models pairs of individuals.

```
class couple{
    friend std::ostream& operator<<(std::ostream& os, const couple& c);

    public:
        couple(); //no parameter method
        couple(const std::string& one, const std::string& two); //input is two strings
        const std::string& getFirst() const; //return C++ string
        const std::string& getSecond() const; //return C++ string

    private:
        std::string first; //private variable -> first name
        std::string second; //private variable -> second name
}

//class implementation
couple::couple(){}
couple::couple(const string& one, const string& two): first(one), second(two){}

const string& couple::getFirst() const { return first; }
const string& couple::getSecond() const {return second; }

ostream& operator<<(ostream& os, const couple& c){ //this is a function
    os << c.first << " and " << c.second;
    return os;
}
```

Here, **std** refers to the C++ standard library. The ampersand (&) means passing by reference and direct modification of the string in a couple object. The full implementation and a sample usage (main method) of the couple class is

```
#include <iostream>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <ostream>

class couple{
    friend std::ostream& operator<<(std::ostream& os, const couple& c);

    public:
        couple(); //no parameter method
        couple(const std::string& one, const std::string& two); //input is two strings
        const std::string& getFirst() const; //return C++ string
        const std::string& getSecond() const; //return C++ string

    private:
        std::string first; //private variable -> first name
        std::string second; //private variable -> second name
};

//class implementation
couple::couple(){}
couple::couple(const std::string& one, const std::string& two): first(one), second(two){}

const std::string& couple::getFirst() const { return first; }
const std::string& couple::getSecond() const {return second; }
```

```

std::ostream& operator<<(std::ostream& os, const couple& c){ //this is a function
    os << c.first << " and " << c.second;
    return os;
}

using namespace std;
int main(int argc, char *argv[]){
    couple hosts("Colleen","Joe");
    couple guests[5]; //works due to zero-argument constructor

    guests[0] = couple("Robin","Jamie");
    guests[1] = couple("April","Chris");
    guests[2] = couple("Jennifer","Andrew");
    guests[3] = couple("Allison","Thomas");
    guests[4] = couple("John","Michael");
    cout << "Dinner party guess list: " << endl;
    cout << "Hosts: " << hosts << endl;
    for (const couple& c: guests){ //works since context provides array length
        cout << "Guests: " << c << endl;
    }
    cout << endl;
    cout << "We're serving duck!" << endl;
    return 0;
}
```

Sets (CS 103)

Computability theory delves into which problems we can solve with a computer. Complexity theory dives into what makes some problems harder to solve than others. Discrete mathematics allows us to be certain we can answer those problems.

Set Theory

A set is an un-ordered collection of distinct objects called elements (no duplicates), which could be possibly other sets. Two sets are equal when they have the same content; order is invariant. By design, $x \neq \{x\}$, as one is a number and the other is a set, or metaphorical box, containing a number.

$$\{1, 2, 3\} = \{3, 2, 1\}$$

The empty set is the set with no elements $\{\}$, written formally as $\varnothing$.

Some sets contain an infinite quantity of numbers, such as integers ($Z = \{\cdots, -1, 0, 1, \cdots\}$) and natural numbers ($N = \{1, 2, 3, \cdots\}$). To describe them succinctly, we can use set builder notation.

$$\{n \mid n \in N \text{ and } n \text{ is even}\}$$

More generally, we use $\boxed{\{x \mid \text{ some property that } x \text{ satisfies}\}}$ or $\boxed{\{x \in S \mid \text{ some property that } x \text{ satisfies}\}}$.

- The union of two sets A and B, $A \cup B$, is all elements in either set.
- The intersection, $A \cap B$ is all elements in *both* sets.
- The difference, $A - B$ or $A \backslash B$, is all elements in A but not in B.

- The symmetric difference, $A\Delta B$ is all elements in either set *but not both*.

A set S is called a subset of T, meaning $S \subset T$, if all elements of S are in T. The power set $\wp(X)$ is the set of all subsets of set X. Note that $\wp(\varnothing) = \{\varnothing\}$.

The cardinality, denoted $|A|$, is the number of elements in that set. We introduce $|N| = \aleph_0$, as there are infinitely many natural numbers (there are different versions of ∞, so we refrain from using that symbol). Two sets, A and B, have the same **cardinality** if there is a way to pair their elements off without leaving any elements uncovered. The below demonstrates why $|N| = |Z|$: there are as many positive integers as all integers.

N	0	1	2	3	4	5	6	$\cdots$
Z	0	-1	1	-2	2	-3	3	$\cdots$

Cantor provided an elegant diagonalization proof for the following result with infinite sets.

Cardinality of Power Set (Cantor's Theorem)

Let X be a set. Then,

$$|X| < |\wp(X)|$$

Proof: We assume that $|X| \geq |\wp(X)|$. There must be a mapping from an element $x_i \in X$ to each element $y_i \in \wp(X)$ for $i \geq 1$. We can represent each y_i as a distinct binary string $b_{1i}b_{2i}b_{3i}\cdots$, where b_{1i} is 1 if y_i contains x_i and 0 otherwise. We form the element $b_{11}b_{22}b_{33}\cdots$ and proceed to invert this binary string into $!(b_{11}b_{22}b_{33}\cdots)$. We note that this new string must belong in $\wp(X)$ by definition of power set but this is not the case since $!b_{11}$ means it cannot be y_1, $!b_{22}$ means it cannot be y_2, and so forth. Thus, not all elements of $\wp(X)$ are covered, implying a contradiction.

The general takeaway is that there is no biggest infinity and that not all infinite sets have the same size.

$$\begin{aligned} |N| &< |\wp(N)| \\ |\wp(N)| &< |\wp(\wp(N))| \\ |\wp(\wp(N))| &< |\wp(\wp(\wp(N)))| \end{aligned}$$

Proof Writing (CS103)

Basic examples of a mathematical proof to practice formal proof writing.

Theorem: If n is an even integer, then n^2 is even.
Proof: Let n be an even integer, meaning that $n = 2k$ for some integer k. Thus, $n^2 = (2k)^2 = 2(2k^2) = 2m$ for integer $m = 2k^2$. Hence, n^2 is even.

Generally, to prove a statement of the design "If P, then Q", assume that P is true, and show that Q is true as well.

Theorem: For any integers m and n, if m and n are odd, then $m + n$ is even.
Proof: Consider any arbitrary integers m and n that are odd. Since m and n are odd, there are integers k and r such that $m = 2k + 1$ and $n = 2r + 1$, so thus

$$m + n = 2k + 1 + 2r + 1 = 2(k + r + 1) = 2s.$$

Hence, there is an integer s, namely $k + r + 1$, such that $m + n = 2s$. This means that $m + n$ is even, as desired.

We make a distinction between universal and existential statements. A universal statement is of the form "for all x, [some property] holds for x". An existential statement is of the form "there is some x where [some property] holds for x; a simple approach for solvency is to find the right x.

Theorem: For any odd integer n, there exists integers r and s where $r^2 - s^2 = n$.
Proof: Pick any odd integer n. Since n is odd, then there exists $k \in Z$ so that $n = 2k+1$. Let $r = k+1$ and $s = k$. Then, as required,

$$r^2 - s^2 = (k + 1)^2 - k^2 = 2k + 1 = n.$$

A good strategy for solving set proofs is to visualize them using venn diagrams.

Theorem: If A, B, and C are sets, then for any $x \in (A \cap B) \cup C$, we have $x \in (A \cup C) \cap (B \cup C)$.
Proof: Consider arbitrary sets A, B, and C; then choose any $x \in (A \cap B) \cap C$. This means that $x \in A \cap B$ or $x \in C$. We examine each case separately.

- *Case 1:* $x \in C$. This means that $x \in A \cup C$ and that $x \in B \cup C$.
- *Case 2:* $x \in A \cap B$. This directly implies that $x \in A$ and $x \in B$, so $x \in A \cup C$ and $x \in B \cup C$.

In either context, we have $x \in A \cup C$ and $x \in B \cup C$, establishing that $x\ in (A \cup C) \cap (B \cup C)$.

Statements (CS103)

Implication

An implication of a statement of the form "If P is true, then Q is true". Implication is directional; the prior statement *does not imply* that "If Q is true, then P is true." For instance, if I told you that a person lives in Anchorage, that means they live in Alaska; but if I told you a person lives in Alaska, that doesn't mean they live in Anchorage – they could also be from Juneau or Fairbanks.

Negation

The negation of a proposition X (either true or false) is a proposition that is true whenever X is false and is false whenever X is true.

- The negation of a universal statement (for all x, $P(x)$ is true) is an existential statement (there exists an x where $P(x)$ is false).
- The negation of an existential statement (there exists x where $P(x)$ is true) is a universal statement (for all x, $P(x)$ is false).
- *The negation of an implication is not an implication.* The negation of "For any x, if $P(x)$ is true, then $Q(x)$ is true" is the statement "there is at least one x where $P(x)$ is true and $Q(x)$ is false."

Contrapositive

The contrapositive of the implication "If P is true, then Q is true" is the implication "If Q is false, then P is false". They are equivalent as they have the same negation. We can prove an implication, thus, by proving its implication.

Theorem: For any $n \in Z$, if n^2 is even. then n is even.
Proof: We leverage proof by contrapositive, showing that if n is odd, then n^2 is odd. This means that $n = 2k + 1$ for some $k \in Z$, so

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1 = 2m + 1.$$

There exists $m \in Z$, namely $2k^2 + 2k$, such that $n^2 = 2m + 1$, so n^2 is odd.

Bi-conditionals

To prove a theorem of the form P if and only if Q, we need to jointly prove that

- If P is true, then Q is true
- If Q is true, then P is true.

Proof by Contradiction

Every statement is either true or false. We can prove a statement P is true by showing that it cannot be false. We first assume that P is false and show that this leads to an impossible contradiction.

Theorem: There is no largest set.
Proof: For the sake of contradiction, assume that this is false, and there is a largest set we call S. By Cantor's Theorem, we have $|S| < |\wp(S)|$ for power set $\wp(S)$. This means that $\wp(S)$ is a larger set, a contradiction as we claimed S was the largest set. Thus, the theorem must be true.

Theorem: For any intteger, if n^2 is even, then n is even.
Proof: For the sake of contradiction, assume that there is an integer n such that n^2 is even but n is odd. Hence, $n = 2k + 1$, meaning that

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1 = 2m + 1.$$

This informs us that n^2 is odd, which is a contradiction since we assumed n^2 to be even. Hence, the assumption was incorrect, so if n^2 is even, then n is also even for integer n.

Joint Probability Density Functions (CS109)

Part of the conceptual package that comes with joint probability density functions are independence, 2-D support, joint PDF, joint CDF, and marginal PDF. Conditional PDFs are also ingrained within the topic. We can discuss how 2 continuous random variables vary with one another, how continuous RV are distributed given evidence, and how continuous RVs can be decomposed into two RVs and vice versa.

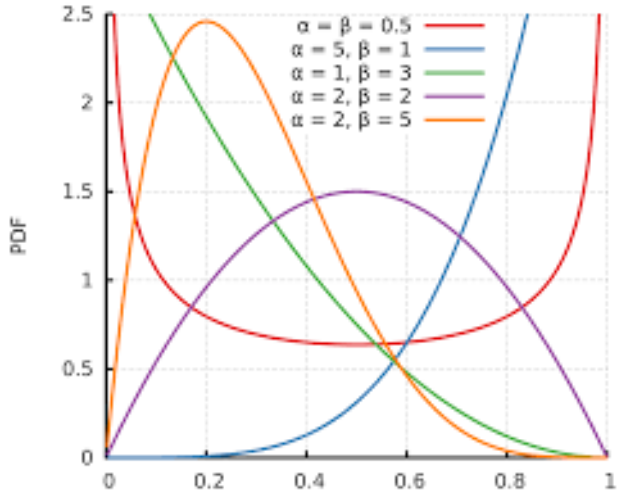
Consider the sum of two independent normals $X_1 \sim N(\mu_1, \sigma_1^2)$ and $X_2 \sim N(\mu_2, \sigma_2^2)$. Supposing independence between X_1 and X_2, we can manufacture the covariance matrix $\Sigma = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$. It turns out that X_1 and X_2 is also normally distributed.

$$X_1 + X_2 \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2).$$

The proof is provided on wikipedia. We can consider the linear transformation of normals: if $Y = aX + b$, then $Y \sim N(a\mu + b, a^2\mu^2)$. If we attempt to find the distribution of $Y = X + X$, we must use the linear transformation approach since X and X are not independent. Thus, we can formalize our understanding by coalescing the two approaches as shown below.

$$aX_1 + bX_2 + c \sim N(a * \mu_1 + b * \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2).$$

Beta Distribution (CS109)



We consider the maximum likelihood estimate (MLE) of p is when flipping a coin with unknown probability. In particular, we think about an experiment where we flip a coin $n + m$ times, resulting in n heads. We are not certain of the probability θ that the coin comes up heads. According to the traditional *frequentist* theory, we have $\theta = \lim_{n+m\to\infty} \frac{n}{n+m} \approx \frac{n}{n+m}$. However, we can actually treat θ as a random variable itself instead of a fixed scalar, which aligns with the Bayesian theory of probability.

Our experiment consists of flipping a coin $n+m$ with an observation of n heads. Let N be the number of heads and θ be the probability of heads. We use a prior of $\theta \sim Uni(0, 1)$, a binomial likelihood of $N \mid \theta = x \sim Bin(n + m, x)$, and a posterior of $f_{\theta|N}(\theta \mid n)$.

$$f_{\theta|N}(x \mid n) = \frac{p_{N|\theta}(n \mid x) f_\theta(x)}{p_N(n)} = \frac{\binom{n+m}{n} x^n (1-x)^m * 1}{p_N(n)} = \frac{1}{c} x^n (1-x)^m.$$

If we flip a coin 8 times and observe $n = 7$ heads and $m = 1$ tails, we have

$$f_{\theta|N}(x \mid n) = \frac{1}{c} x^7 (1-x)^1$$

This resembles a Beta distribution with parameters $a, b > 0$. Here, $B(a, b)$ serves as the normalizing constant and we have

$$X \sim Beta(a, b) \text{ and } f(x) = \frac{1}{B(a,b)} x^{a-1} (1-x)^{b-1}$$

It turns out the posterior belief of the probability θ is a Beta distribution with parameters $a = 8$ and $b = 2$. We can look at the graph of the Beta distribution and find the *maximum a posteriori* (MAP)

estimate by pinpointing its mode. One application for rejection sampling and Bayesian probability theory is CAT (catastrophe) modeling.

We consider conjugate distributions. In particular, we say that Beta is a **conjugate distribution** for Bernoulli. This means that if our prior belief of the parameter is Beta, and our experiment is Bernoulli, then our posterior is also Beta. We can set the prior to reflect how biased we think the coin is *a priori*. We set $\theta \sim Beta(a, b)$ if we have set $a + b - 2$ imaginary trials with $a - 1$ heads and $b - 1$ tails.
Finally, we consider a medical application of the Beta concept. Suppose that the medicine is believed to work 80% of the time. The medical is tried on 20 patients with a success on 14 patients. The prior distribution of θ resembles either $Beta(80 + 1, 20 + 1)$ or $Beta(4 + 1, 1 + 1)$. The posterior distribution will be $\theta \sim Beta(5 + 14, 2 + 6) = Beta(19, 8)$ using the prior of $Beta(5, 2)$.

The expected value of a Beta distribution is equal to $\frac{a}{a+b}$. The mode and maximum value of the distribution is $\frac{a-1}{a+b-2}$. This obtained by setting the gradient of $Beta(a, b)$ equal to 0.

Intro to Machine Learning (CS229)

According to Arthur Samuel in 1959, *machine learning* is a field of study that hands computers the ability to learn without explicit programming.

Tom Mitchell (1998) argues that in a *well-posed learning problem*, a computer program is said to learn from experience E with respect to some task T and some performance measured by P, if its performance on T, as measured by P, improves with experience E.

One of the four major branches of machine learning is *supervised learning*, which encompasses 99% of economic value generated with ML. Supervised learning revolves around $x \to y$ mappings. For instance, when provided with a training set (x_i, y_i), the program will learn to fit a line or function $h(x)$ for mapping an input feature x into an output label y.

A regression problem is a subset of supervised learning where y is a continuous and not discrete value. Another prominent category of supervised learning problems is a *classification problem*, where the prediction variable y is discrete. An example is a breast cancer diagnostic task, where y is either 0 for benign or 1 for malignant. Other tasks in supervised learning include email $\to$ spam, English $\to$ French, audio $\to$ text transcription, and satellite imagery/data $\to$ methane emissions for climate science. In the sphere of corporate industry, companies use (ad, user) to predict whether the user will click on an advertisement or not.

Regression (CS229)

Housing Prices

The objective is to find a regression line that c

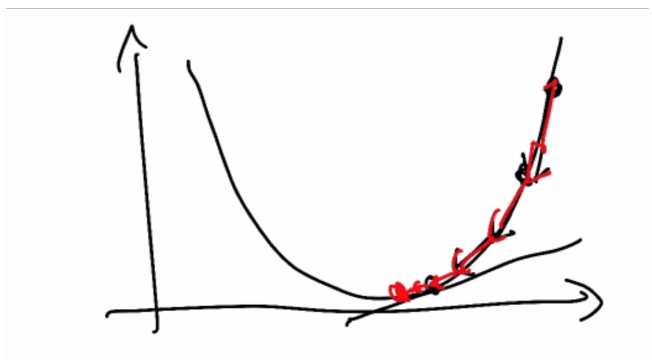
Stochastic Gradient Descent

We use stochastic gradient descent if m is large. The overall schematic of the algorithm is that we repeat for $i = 1$ to m, we update update the weight θ_{ji} for every j to $\theta_j - \alpha * (h_\theta(x^i) - y^i) * x^i_j$.

Loss Landscape

In the loss landscape, we can take a contour map of the colossal surface of the loss function $J(\theta)$. In this contour map, we essentially move down the rings of elevation in steps perpendicular to the contours. When we have a high learning rate, the steps are more large, which injects a high degree of variability into the learning. When he have a slow learning rate, the steps are smaller and more graduated. When using the *stochastic gradient descent*, the movements and parameter adjustments can be quite jittery and seem devoid of methodology, but the movements will eventually gravitate toward a local minimum.

A significant debate is the learning rate α to be deployed. For batch gradient descent, it's not necessary to decrease the learning rate over training. This is because the gradient descent step is naturally decreased since the slope will automatically decrease as we approach a global minimum. Thus, as the slope is smaller, the gradient will smaller, so the algorithm will take smaller and smaller steps.



Kernel Method (CS229)

Suppose that we want to fit a model to our data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ with nonlinear tendencies using linear techniques. Our goal is to identify parameters θ_i such that $h_\theta(x) = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x^1 + \theta_0$. We can first transform the data using a function $\phi(x) : R \to R^4$ that maps a single number to a four dimensional vector $\begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$. Thus,

$$h_\theta(x) = \begin{bmatrix} \theta_0 & \theta_1 & \theta_2 & \theta_3 \end{bmatrix} * \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

We note that $h_\theta(x)$ is linear in θ and $\phi(x)$. The new dataset has then been transformed into the form $\{(\phi(x^{(i)}, y^{(i)})\}_{i=1}^n$. Thus, a cubic polynomial that would have been fit for the old dataset has been transfigured into a linear model for the new, feature dataset. As terminology, we typically call $\phi(x)$ the feature, x the attribute, and the mapping $\phi : R^d \to R^p$ the feature map. Thus, the objective is to minimize the LMS of the new dataset, or find the parameters θ such that we

$$\text{minimize} \quad \frac{1}{2} \sum_{i=1}^{n} (y^{(i)} - \theta^T \phi(x^{(i)})).$$

To perform gradient descent on this new dataset, we loop through the dataset and compute

$$\theta \leftarrow \theta + \alpha * \sum_{i=1}^{n} (y^{(i)} - \theta^T \phi(x^{(i)})) * \phi(x^{(i)})$$

We consider the action we take when p is very large, meaning that the feature map is very high dimensional. When $d > 1$, we still want to fit a cubic polynomial. For instance,

$$\phi(x) = \begin{bmatrix} 1 & x_1 & x_2 & x_3 \cdots x_d & x_1^2 & x_1 x_2 & x_1 x_3 & \cdots & x_2^2 \cdots x_2 x_d & \cdots x_d^2 \cdots x_d^3 \end{bmatrix}$$

Thus, $\theta^T \phi(x) = _ * 1 + _ * x_1 + _ * x_2 + \cdots + _ * x_i x_j + \cdots$, suggesting that every degree-3 polynomial can be written as $\theta^T \phi(x)$. Thus, even though $\phi(x)$ is of very high dimension, we can write $p = 1 + d + d^2 + d^3 = O(d^3)$. Unfortunately, for colossal numbers such as $d = 10^3$ and $p = 10^9$, the run-time for one iteration of GD is $O(np)$.
The key observation is that if θ is initialized at 0, then at any time θ can be represented as $\theta = \sum_{i=1}^{n} \beta_i * \phi(x^{(i)})$ for some $\beta_1, \cdots, \beta_n \in R$. We can prove this using mathematical induction over time. The base case is iteration= 0, in which $\theta = 0 = \sum_{i=1}^{n} 0 * \phi(x^{(i)})$. We find that over iteration 1,

$$\begin{aligned} \theta \leftarrow \theta + \alpha * \sum_{i=1}^{n} (y^{(i)} - \theta^T \phi(x^{(i)})) * \phi(x^{(i)}) \\ = 0 + \alpha * \sum_{i=1}^{n} (y^{(i)}) * \phi(x^{(i)}) \\ = \sum_{i=1}^{n} (\alpha * y^{(i)}) * \phi(x^{(i)}). \end{aligned}$$

Additionally, we find that over the next iteration:

$$\begin{aligned} \theta \leftarrow \theta + \alpha * \sum_{i=1}^{n} (y^{(i)} - \theta^T \phi(x^{(i)})) * \phi(x^{(i)}) \\ = \sum_{i=1}^{n} (\alpha * y^{(i)}) * \phi(x^{(i)}) + \alpha * \sum_{i=1}^{n} (y^{(i)} - \theta^T \phi(x^{(i)})) * \phi(x^{(i)}) \\ = \sum_{i=1}^{n} (\beta_i + \alpha * (y^{(i)} - \theta^T \phi(x^{(i)})) * \phi(x^{(i)}). \end{aligned}$$

We note that the term written in red can be considered a new β_i as it is a constant.